Cross-Reality Project — Status Report

Platform: Meta Quest 3

Engine: Unity 6000.3.8f1 URP

SDK: Meta XR 201.0.0

1. Project Overview

This project implements a **bidirectional Mixed Reality ↔ Desktop pipeline** running on the Meta Quest 3.

The user looks at a real-world object through the passthrough camera, presses a button, and an AI-generated 3D model of that object appears in their mixed-reality space—grabbable, physics-enabled, and interactable. They can then send it to a laptop where it opens in Blender. After editing, the updated model can be sent back into VR. The desktop can also independently push any `.glb` file into the headset.

2. System Architecture

The following diagram illustrates the data flow between the Quest 3, Firebase, and the Desktop environment:

```
┌──────────────────────────────────────────────────────────────┐
│                      META QUEST 3 (MR)                       │
│                                                              │
│  ObjectDetectionAgent ──► SAM3DCaptureManager                │
│    (on-device CV model)          │                           │
│    detects: cup, chair, mouse…   │                           │
│                                  ▼                           │
│  CameraCapture ──────────► FalAiClient ──────► FAL AI Cloud  │
│  (passthrough / screen           │               SAM-3D model │
│   capture fallback)              │               generation   │
│                                  ▼                           │
│               GLB URL ──► GLBModelLoader                     │
│                                  │                           │
│                                  ▼                           │
│          Grabbable 3D Model (GLTFast + Interaction SDK)      │
│          BoxCollider · Rigidbody · HandGrabInteractable      │
│                                  │                           │
│                     ┌────────────┴────────────┐              │
│                     │                         │              │
│              Press B button          TransferZone            │
│                     │               (collision-based,        │
│                     ▼                optional)               │
│             DatabaseManager                                  │
│           (Firebase REST POST)                               │
```

```
│                    │                                          │
│  VRModelReceiver ←── polls /desktop_to_vr every 3s           │
│  (spawns desktop models                                      │
│   to the right of capture models)                            │
└──────────────────────────────────────────────────────────────┘
                     │                      ▲
           /models (legacy)          /desktop_to_vr
           /vr_to_desktop                    │
                     │                       │
┌──────────────────────────────────────────────────────────────┐
│               FIREBASE REALTIME DATABASE                     │
│      [https://crossreality-ec12f.asia-southeast1.firebasedatabase](https://crossr
│                                                              │
│   /models          →  VR-to-Desktop  (legacy collection)     │
│   /desktop_to_vr   →  Desktop-to-VR  (new collection)        │
└──────────────────────────────────────────────────────────────┘
                     │                       │
              receiver.py              upload_to_vr.py
              (polls + opens           blender_send_to_vr.py
               Blender)                (Blender N-panel addon)
                     │                       │
┌──────────────────────────────────────────────────────────────┐
│                     LAPTOP (Windows)                         │
│                                                              │
│  receiver.py           — auto-opens GLBs in Blender          │
│  upload_to_vr.py       — CLI uploader, any .glb → VR         │
│  blender_send_to_vr.py — Blender addon, "Send to VR" button  │
└──────────────────────────────────────────────────────────────┘
```

3. Complete Bidirectional Loop

1. **Capture:** User presses 'A' button → Passthrough frame captured → AI generates textured GLB via FAL AI.
2. **Interaction:** VR Model spawns in MR space → Physics-enabled and grabbable with hands or controllers.
3. **Transfer:** User presses 'B' button → URL sent to Firebase `/models`.
4. **Desktop Reception:** `receiver.py` polls Firebase → Automatically downloads and opens model in Blender.
5. **Edit & Return:** User edits in Blender → "Send to VR" via N-panel → Model uploaded and pushed to `/desktop_to_vr`.
6. **VR Reception:** `VRModelReceiver.cs` polls every 3s → Updated model spawns in the headset.

4. Unity Scripts (Quest 3)

- **SAM3DCaptureManager.cs:** Orchestrates the capture workflow and selects labels for AI

prompts.

- **CameraCapture.cs:** Captures passthrough camera frames or falls back to screenshots on older firmware.
- **FalAiClient.cs:** Handles communications with the FAL AI SAM-3D API.
- **GLBModelLoader.cs:** Downloads and instantiates models with physics and grabbability.
- **DatabaseManager.cs:** Firebase Realtime Database REST client.
- **VRModelReceiver.cs:** Continuous polling agent to receive models from the desktop.

5. Dependencies & Packages

Package	Version	Purpose
`com.meta.xr.sdk.core`	201.0.0	OVRCameraRig, Passthrough Access
`com.meta.xr.sdk.interaction`	201.0.0	Grabbable, HandGrabInteractable
`com.unity.cloud.gltfast`	6.10.1	GLB/GLTF loading at runtime
`com.unity.ai.inference`	2.3.0	On-device ML for object detection
`com.unity.render-pipelines.universal`	17.3.0	URP rendering pipeline

6. Confirmed Working Features

- **Object Detection:** Logged sightings of `person`, `keyboard`, `chair`, and `mouse`.
- **Model Generation:** 8+ successful unique GLB generations.
- **Bi-directional Pipeline:** End-to-end confirmation from VR to Blender and back.
- **Hand Tracking:** Pinch and palm grab verified for generated assets.

7. Known Limitations

- **HzOS Versioning:** Raw camera access requires v85; earlier versions use screenshot fallback.
- **Security:** Firebase secrets are plaintext (standard for project prototypes).
- **File Lifespan:** Models uploaded from desktop to 0x0.st expire after 30 days.